27. Stack in Java Collections — Quick Notes

What is Stack?

- Stack follows **LIFO (Last In First Out)** principle.
- Last inserted element is removed first.

Example:
Push: 10, 20, 30

Top
↓
30
20
10

Pop → 30 removed

Stack Implementations in Java

Class	Hierarchy	Recommended
`Stack<E>`	`Stack → Vector → List`	❌ No
`ArrayDeque<E>`	`ArrayDeque → Deque → Queue`	✅ Yes

Which One Should We Use?

`ArrayDeque`

- Preferred implementation for Stack.
- Faster in **single-threaded** environments.
- Not synchronized.
- Less overhead.

`Stack`

- Legacy class.
- Extends `Vector`.
- Thread-safe (synchronized).
- Extra overhead due to synchronization.

> **Recommendation:** Use `ArrayDeque` instead of `Stack`.

Creating a Stack

Using ArrayDeque (Recommended): Deque<Integer> stack = new ArrayDeque<>();
Using Stack (Legacy): Stack<Integer> stack = new Stack<>();

Stack Operations

Push

Adds element to top.

```
stack.push(10);
stack.push(20);
stack.push(30);
```

Stack:

Top
↓
30
20
10

Peek

Returns top element **without removing** it.

```
stack.peek();
```

Output: 30

Stack remains: 30 20 10

Pop

Returns and removes top element.

```
stack.pop();
```

Output: 30

Stack becomes:
Top
↓
20
10

Example

```
Deque<Integer> stack = new ArrayDeque<>();

stack.push(10);
stack.push(20);
stack.push(30);

System.out.print(stack.peek());
System.out.print(stack.pop());
System.out.print(stack.peek());
```

Output: 30 30 20

Utility Methods

Size

Returns number of elements.

```
stack.size();
```

Example: 10 20 30

size() → 3

---

isEmpty

Checks whether stack is empty.
stack.isEmpty();

Output: false

Time Complexity

Operation	Complexity
`push()`	O(1)*
`pop()`	O(1)*
`peek()`	O(1)
`size()`	O(1)
`isEmpty()`	O(1)

* Amortized O(1)

Why Amortized O(1)?

Both `ArrayDeque` and `Vector` use **dynamic arrays** internally.

When array becomes full: Old Array
[10 20 30 40]

↓ Resize

New Array
[10 20 30 40 _ _ _ _]

Resizing takes: O(n)

However, Most insertions do **not** resize.
Therefore: Average cost of push() = O(1)
This is called: Amortized O(1)

Hierarchy

Stack

Collection
↓
List
↓
Vector
↓
Stack

ArrayDeque

Collection
↓
Queue
↓
Deque
↓
ArrayDeque

Stack vs ArrayDeque

Feature	Stack	ArrayDeque
Legacy Class	Yes	No
Thread-safe	Yes	No
Synchronization Overhead	Yes	No
Recommended	❌	✅
Internal Structure	Dynamic Array (Vector)	Dynamic Array
push/pop complexity	O(1)*	O(1)*

Important Interview Points

- Stack follows **LIFO**.
- `peek()` **does not remove** the top element.
- `pop()` **returns and removes** the top element.
- `ArrayDeque` is preferred over `Stack`.
- `Stack` is a **legacy class** extending `Vector`.
- `push()` and `pop()` are **amortized O(1)** due to dynamic resizing.
- Use `Deque` reference type with `ArrayDeque` implementation.

Deque<Integer> stack = new ArrayDeque<>();

Quick Revision

Concept	Remember
Principle	LIFO
Recommended Class	`ArrayDeque`
Legacy Class	`Stack`
View Top	`peek()`
Remove Top	`pop()`
Add Top	`push()`
Empty Check	`isEmpty()`
Size	`size()`
Complexity	O(1)*
* Means	Amortized O(1)